

Reviewer Pack — Toric Variety Degree Inverse to SOS Refutation Degree for 3-...

Entry ee97068481df · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 04:58:29 UTC

Toric Variety Degree Inverse to SOS Refutation Degree for 3-SAT

Entry ID: ee97068481df **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 04:58:29 UTC

1. Conjecture

Field A (mathematical branch): Toric Geometry **Field B** (complexity object): SOS Refutation Degree for CSPs

Statement:

For a 3-SAT instance with n variables, the degree of the toric variety defined by its clauses is inversely proportional to the minimal SOS refutation degree. Specifically, $\deg(V) = \Theta(1 / \text{refutation_degree}(n))$ for $n \leq 40$.

Rationale (proposer's reasoning):

Toric varieties encode combinatorial constraints of clauses as algebraic varieties. Their degree, a measure of complexity, may mirror SOS refutation requirements, revealing geometric structure in CSP hardness.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 17379fd5d036f555

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random

def generate_3sat_instance(n, m):
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause = random.sample(variables + [-v for v in variables], 3)
        clauses.append(clause)
    return clauses

def monomial_degree(clause):
    return len(clause)

def toric_variety_degree(clauses):
    degrees = [monomial_degree(clause) for clause in clauses]
    return sum(degrees)

def sos_refutation_degree(n):
    # Placeholder function, replace with actual SOS refutation degree calculation
    return n # Simplified example

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    m = 2 * n # Number of clauses

    clauses = generate_3sat_instance(n, m)
    degree = toric_variety_degree(clauses)
    refutation_degree = sos_refutation_degree(n)

    metric_value = degree * refutation_degree
    conjecture_holds = abs(metric_value - 1) < 1e-6

    return {
        "metric_name": "degree_product",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"n={n}, degree={degree}, refutation_degree="
```

```

    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 50)) # Default to first 30 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next(r["counterexample"] for r in results if r["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

gree=240, refutation_degree=40'}
TRIAL: {'metric_name': 'degree_product', 'metric_value': 9600, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'degree_product', 'metric_value': 9600, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'degree_product', 'metric_value': 9600, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'degree_product', 'metric_value': 9600, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'degree_product', 'metric_value': 9600, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'degree_product', 'metric_value': 9600, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'degree_product', 'metric_value': 9600, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'degree_product', 'metric_value': 9600, 'instances_tested': 1, 'conjecture_holds': True}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0592bc1b.py", line 77, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0592bc1b.py", line 77, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~~
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

parse_fail | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111522
2	propose	ollama_remote	qwen3:8b	0	0	107939
3	propose	ollama_remote	qwen3:8b	0	0	111840
4	propose	ollama_remote	qwen3:8b	0	0	106306
5	preregistration	ollama_remote	qwen3:8b	0	0	25472
6	novelty	ollama_remote	qwen3:8b	0	0	22050
7	novelty	ollama_remote	qwen3:8b	0	0	13600
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13048
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7292
10	judge	ollama_remote	qwen3:8b	0	0	23304

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 542374 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/ee97068481df.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/ee97068481df.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/ee97068481df.tar.gz (if generated)